Memoria técnica del proyecto

Reconocedor y clasificador de caracteres en tiempo real 2D/3D mediante Cadenas Ocultas de Markov (HMM)

Procesos Estocásticos (603504) · Ingeniería de Sistemas · Universidad de los Llanos
Estado al 6 de noviembre de 2026 · Modo 2D funcional · Repositorio: github.com/lopezand2701/Estocasticos

1. Resumen: qué hace el proyecto y en qué estado está
2. Todo lo que hemos hecho (cronología)
3. Arquitectura: archivos y para qué sirve cada uno
4. Temas del curso que usa el proyecto
5. Fórmulas utilizadas, una por una (qué hace y cómo está compuesta)
6. Ejemplo resuelto a mano (letra "L")
7. Parámetros elegidos y resultados obtenidos
8. Lo que falta
9. Cómo ejecutar el programa en un computador nuevo (solo con Python)

1. Resumen

Qué hace: una aplicación de escritorio donde el usuario dibuja una letra minúscula del español (27 letras, incluida la ñ) con el mouse. Mientras dibuja, el programa calcula para cada letra la probabilidad de que sea esa, las muestra ordenadas de mayor a menor, y dibuja con línea punteada cómo cree que se va a terminar la letra (autocompletado). Debe funcionar en un plano (2D) y sobre una esfera (3D).

Cómo lo hace: cada letra es una **Cadena Oculta de Markov** $\lambda = (A, B, \pi)$: los estados ocultos son las *fases* del trazo y las observaciones son las *direcciones* en que se mueve el mouse. Los modelos se entrenan antes (offline) con Baum-Welch a partir de miles de dibujos reales; la app usa Forward para clasificar, Bayes para convertir a porcentajes y Viterbi + duraciones para proyectar.

Restricción del profesor: todo el HMM se programó desde cero; solo se usa NumPy para operar con arreglos. No se usa hmmlearn, scikit-learn, scipy, PyTorch ni TensorFlow.

Componente	Estado
Motor HMM (Forward, Backward, Viterbi, Baum-Welch, escalamiento)	**Hecho y probado (13 tests)**
Extracción de características 2D (remuestreo + código de cadena + lápiz arriba)	**Hecho**
Entrenamiento de las 27 letras con el dataset del curso	**Hecho · 88.6 % de acierto (top-1), 95.4 % (top-3)**
Interfaz: menú + reconocedor 2D con ranking y proyección	**Hecho**
Modo 3D (esfera)	**Pendiente**
Documentación final, ZIP y sustentación	**Pendiente**

2. Todo lo que hemos hecho

Fecha	Qué se hizo
30 oct	Lectura del enunciado y plan de 3 semanas. Explicación del proyecto y de las decisiones técnicas.
30 oct	Día 1: estructura de carpetas, entorno virtual `.venv`, `requirements.txt`, `.gitignore`, README, `docs/decisiones.md`. Repositorio en GitHub (se resolvieron los errores 403 de credenciales y el uso de token).
30 oct	Guía de estudio en español del tutorial de Rabiner (1989): `docs/Guia_Rabiner_ES.pdf`.
3–4 nov	Análisis del repositorio *Glifo* de un compañero: página web compartida por todo el curso para recolectar dibujos. Se aportaron las 27 letras. Formato de datos: puntos [x, y, t] en un cuadrado 0–1000 con la y hacia abajo.
4 nov	Día 2: ejemplo de Forward, fuerza bruta, Viterbi, Bayes y escalamiento resuelto a mano (letras "L" y "Γ"). Documento `docs/Proyecto_Explicado_Facil.pdf`.
5 nov	Código completo del modo 2D: motor HMM, extractor de características, reconocedor, entrenamiento, evaluación, interfaz y tests.
6 nov	Entrenamiento con el dataset real (5 711 muestras): 82.6 %. Mejora con la **probabilidad de terminar** η: 88.6 %. Corrección del menú para pantallas con escala 125–150 %.

3. Arquitectura

El enunciado pide separar tres capas: matemática (`HMMEngine`), procesamiento geométrico (`FeatureExtractor`) e interfaz (`GUI`).

Archivo	Capa	Responsabilidad
`src/hmm/HMMEngine.py`	Matemática	Clase `HMM`: λ = (A, B, π), topología izquierda-derecha, Forward/Backward escalados, paso Forward incremental, Viterbi, Baum-Welch, η, duraciones, guardar/cargar JSON.
`src/hmm/Recognizer.py`	Matemática	Usa los 27 modelos: actualiza α con cada símbolo nuevo, aplica Bayes (ranking) y calcula la proyección.
`src/features/FeatureExtractor.py`	Geometría	Remuestreo por longitud de arco, cuantización en 16 direcciones, símbolos de "lápiz arriba".
`src/gui/MenuView.py`	Interfaz	Ventana A: menú de inicio (Modo 2D / Modo 3D).
`src/gui/View2D.py`	Interfaz	Ventana B: une lienzo, reconocedor y panel; histéresis de la proyección.
`src/gui/Canvas2D.py`	Interfaz	Lienzo cuadrado 0–1000 con las mismas guías de Glifo; captura del mouse; dibuja trazo y proyección.
`src/gui/ProbabilityPanel.py`	Interfaz	Lista de 27 letras: ícono, barra y porcentaje.
`src/gui/MainWindow.py`, `styles.py`	Interfaz	Ventana principal (cambia entre vistas) y estilo oscuro minimalista.
`training/cargar_datos.py`	Offline	Lee el JSON de Glifo (2 formatos), filtra muestras inválidas, separa entrenamiento/prueba por autor.
`training/train.py`	Offline	Entrena los 27 HMM con Baum-Welch y guarda `models/2d/hmm_2d.json`.
`training/evaluate.py`	Offline	Precisión top-1/top-3, precisión según % del trazo, confusiones.
`tests/test_hmm.py`, `test_features.py`	Pruebas	13 pruebas automáticas (incluyen los números calculados a mano).
`main.py`	—	Arranca la aplicación.

```
dibujo (puntos x, y) → FeatureExtractor → símbolos O₁…Oₜ → Recognizer (Forward + Bayes) → panel de porcentajes
                                                                                  └↳ fase actual + duraciones → proyección punteada
```

4. Temas del curso utilizados

Tema	Descripción breve	Dónde se usa
Proceso estocástico	Sistema que evoluciona en el tiempo de forma aleatoria; secuencia de variables aleatorias X_1, X_2, …	El trazo de una letra: nadie la dibuja igual dos veces.
Probabilidad condicional	P(A | B): probabilidad de A sabiendo que ocurrió B.	P(O | λ_c): probabilidad de las flechas si la letra es c.
Teorema de Bayes	Invierte una condicional: P(A|B) = P(B|A)P(A)/P(B).	Pasar de P(O | letra) a P(letra | O) para el ranking.
Cadena de Markov / propiedad de Markov	El siguiente estado depende solo del actual, no de la historia.	Las fases de la letra.
Matriz de transición (estocástica)	a_ij = P(ir a j | estoy en i); cada fila suma 1.	Quedarse en una fase o avanzar.
Distribución inicial	π_i = P(empezar en i).	π = (1, 0, …, 0).
Estado absorbente; cadena izquierda-derecha	Absorbente: no se sale de él. Izquierda-derecha: no se retrocede.	Fases que solo avanzan; la última es absorbente.
Tiempo de permanencia (geométrica)	Pasos seguidos en un estado; media 1/(1 − a_ii).	Cuánto dibujar de cada fase en la proyección.
Cadena Oculta de Markov (HMM)	Cadena de Markov no observable que emite observaciones; λ = (A, B, π).	Un modelo por letra.
Matriz de emisión	b_j(k) = P(observar v_k | estado j).	Qué direcciones son típicas en cada fase.
Verosimilitud	P(datos | modelo): qué tan bien explica el modelo los datos.	Comparar las 27 letras.
Algoritmo Forward / Backward	Programación dinámica para P(O | λ) en $O(N^2T)$ en vez de O(T·N^T).	Clasificación (Forward) y entrenamiento (ambos).
Algoritmo de Viterbi	Secuencia de estados más probable (máximo en vez de suma + retroceso).	Saber cuántos pasos lleva la fase actual.
Baum-Welch (EM, máxima verosimilitud)	Reestima A y B con conteos esperados; la verosimilitud nunca baja.	Entrenamiento offline.
Predicción	Propagar la cadena hacia el futuro desde el estado actual.	Autocompletado (línea punteada).
Apoyo: cuantización, remuestreo, logaritmos, POO	Discretizar señales continuas; evitar desbordamientos; organizar el código.	FeatureExtractor, escalamiento, arquitectura.

5. Fórmulas utilizadas

Notación general: **N** = número de estados (fases), **M** = número de símbolos (32), **T** = longitud de la secuencia observada, **O = O₁ O₂ … O_T** = secuencia de símbolos, **q_t** = estado oculto en el instante t, **λ = (A, B, π)** = modelo de una letra. Las ecuaciones numeradas (ec. 19, 20…) son las de Rabiner (1989).

5.1 Modelado

F1 Propiedad de Markov

```
P(q_{t+1} = S_j | q_t = S_i, q_{t−1}, …, q_1) = P(q_{t+1} = S_j | q_t = S_i) = a_ij
```

Qué hace: dice que para predecir la próxima fase solo importa la fase actual.

Componentes: q_t = fase en el paso t; S_i, S_j = estados posibles; a_ij = probabilidad de pasar de i a j.

En el código: es la razón por la que A es una matriz N×N y Forward solo mira el paso anterior.

F2 El modelo λ = (A, B, π) y sus restricciones

```
A = [a_ij],  a_ij = P(q_{t+1}=S_j | q_t=S_i),     Σ_j a_ij = 1
B = [b_j(k)], b_j(k) = P(O_t = v_k | q_t = S_j),   Σ_k b_j(k) = 1
π = [π_i],   π_i = P(q_1 = S_i),                  Σ_i π_i = 1
```

Qué hace: define completamente una letra. A = cómo avanzan las fases, B = qué direcciones produce cada fase, π = dónde empieza.

Componentes: v_k = símbolo k (dirección); las sumas = 1 porque cada fila es una distribución de probabilidad.

En el código: atributos self.A (N×N), self.B (N×32), self.pi (N) de la clase HMM.

F3 Topología izquierda-derecha (Bakis) con Δ = 2

```
a_ij = 0   si  j < i   o   j > i + Δ          π = (1, 0, …, 0)          a_NN = 1
Inicial:  a_ii = s,   a_ij = (1 − s)/(nº de destinos − 1)  para i < j ≤ i+Δ
```

Qué hace: obliga a que las fases solo avancen (quedarse, avanzar 1 o saltar 1). La última fase es absorbente.

Componentes: Δ = salto máximo (2); s = probabilidad inicial de quedarse (0.6, luego se ajusta con F15).

Propiedad clave: si a_ij empieza en 0, Baum-Welch lo deja en 0 (porque ξ_t(i,j) = 0), así la estructura se conserva sola.

En el código: HMM._left_right_A().

5.2 Del dibujo a los símbolos

F4 Remuestreo por longitud de arco

```
Se recorre la polilínea p_0, p_1, … y se colocan puntos r_0 = p_0, r_1, r_2, … tales que
‖r_{n+1} − r_n‖ medido a lo largo del camino = STEP = 25   (en el espacio 0..1000)
Punto interpolado:  r = p_a + (p_b − p_a) · (resto / ‖p_b − p_a‖)
```

Qué hace: deja los puntos a la misma distancia, así no importa si el usuario dibujó rápido (pocos puntos) o lento (muchos puntos).

Componentes: p_a, p_b = extremos del segmento actual; resto = distancia que falta para el próximo punto; STEP = 1/40 del lienzo.

Propiedad: al agregar puntos al final, los ya calculados no cambian (necesario para el Forward incremental).

En el código: FeatureExtractor.resample().

F5 Cuantización angular (código de cadena, K = 16)

```
θ = atan2(−Δy, Δx)  mod 2π
k = round( θ / (2π/K) )  mod K          (sectores de 22.5°)
Símbolo:  O_t = k   (lápiz abajo)      O_t = k + 16   (salto con el lápiz arriba)
```

Qué hace: convierte cada desplazamiento entre puntos remuestreados en una de 16 flechas. Los saltos entre trazos (p. ej., del palito al punto de la "i") usan los símbolos 16–31.

Componentes: Δx, Δy = desplazamiento; el signo menos en Δy corrige que en pantalla la y crece hacia abajo; k = 0 es →, 4 es ↑, 8 es ←, 12 es ↓. Alfabeto total M = 32.

En el código: FeatureExtractor.quantize() y strokes_to_symbols().

5.3 Problema 1: evaluación

F6 Cálculo directo (fuerza bruta) · ecs. 13–17

```
P(O | Q, λ) = b_{q1}(O_1) · b_{q2}(O_2) ⋯ b_{qT}(O_T)
P(Q | λ)    = π_{q1} · a_{q1 q2} · a_{q2 q3} ⋯ a_{q(T−1) qT}
P(O | λ)    = Σ_{todas las Q}  P(O | Q, λ) · P(Q | λ)
```

Qué hace: suma la probabilidad de todos los caminos de estados posibles.

Componentes: Q = q_1…q_T un camino; hay N^T caminos → costo ≈ 2T·N^T (imposible para T grande).

En el código: solo en tests/test_hmm.py (fuerza_bruta) para comprobar Forward.

F7 Algoritmo Forward · ecs. 18–21

```
Definición:     α_t(i) = P(O_1 O_2 ⋯ O_t, q_t = S_i | λ)
Inicialización: α_1(i) = π_i · b_i(O_1)
Inducción:      α_{t+1}(j) = [ Σ_{i=1..N} α_t(i) · a_ij ] · b_j(O_{t+1})
Terminación:    P(O | λ) = Σ_{i=1..N} α_T(i)
```

Qué hace: calcula P(O | λ) acumulando, paso a paso, cuánta probabilidad llega a cada estado. Costo N^2T.

Componentes: α_t(i) = probabilidad de haber visto las primeras t flechas y estar en la fase i; Σ α_t(i)·a_ij = "lo que llega" a j; b_j(O_{t+1}) = "lo que emite" j.

En el código: HMM.forward().

F8 Escalamiento (evita que las probabilidades lleguen a 0)

```
ᾱ_t(j) = [ Σ_i α̂_{t−1}(i) a_ij ] · b_j(O_t)        (paso normal con los α escalados)
c_t     = 1 / Σ_j ᾱ_t(j)                            (coeficiente de escala)
α̂_t(j) = c_t · ᾱ_t(j)                               (ahora Σ_j α̂_t(j) = 1)
log P(O | λ) = − Σ_{t=1..T} log c_t
```

Qué hace: normaliza α en cada paso. Sin esto, con T ≈ 100 el producto de probabilidades se vuelve 0 en el computador (underflow).

Componentes: c_t = factor guardado; α̂_t = α normalizado (distribución de la fase actual dado lo visto); la verosimilitud se recupera sumando logaritmos.

En el código: HMM.forward() devuelve (α̂, c); HMM.log_likelihood().

F9 Forward incremental (tiempo real)

```
α̂_t = normalizar( (α̂_{t−1} · A) ⊙ B[:, O_t] )
log P(O_1:t | λ) = log P(O_1:t−1 | λ) + log Σ_j ᾱ_t(j)
```

Qué hace: cuando llega una flecha nueva, aplica un solo paso de F7/F8 en vez de recalcular todo. Por eso el ranking se actualiza sin retraso.

Componentes: ⊙ = producto elemento a elemento; B[:, O_t] = columna de emisión del símbolo nuevo.

En el código: HMM.forward_step() y Recognizer.update().

F10 Algoritmo Backward escalado · ecs. 23–25

```
Definición:     β_t(i) = P(O_{t+1} ⋯ O_T | q_t = S_i, λ)
Inicialización: β̂_T(i) = c_T                (β_T = 1, escalado)
Inducción:      β̂_t(i) = c_t · Σ_j a_ij · b_j(O_{t+1}) · β̂_{t+1}(j)
```

Qué hace: lo mismo que Forward pero desde el final hacia atrás. Se usa solo en el entrenamiento.

Componentes: se escala con los mismos c_t de Forward para que α̂·β̂ dé directamente γ.

En el código: HMM.backward().

5.4 Problema 2: decodificación

F11 Algoritmo de Viterbi (en logaritmos) · ecs. 30–35

```
Definición:     δ_t(i) = max_{q1…q(t−1)} P(q_1 ⋯ q_t = i, O_1 ⋯ O_t | λ)
Inicialización: δ_1(i) = log π_i + log b_i(O_1),        ψ_1(i) = 0
Recursión:      δ_t(j) = max_i [ δ_{t−1}(i) + log a_ij ] + log b_j(O_t)
                ψ_t(j) = argmax_i [ δ_{t−1}(i) + log a_ij ]
Terminación:    log P* = max_i δ_T(i),     q*_T = argmax_i δ_T(i)
Retroceso:      q*_t = ψ_{t+1}(q*_{t+1}),   t = T−1, …, 1
```

Qué hace: encuentra el único camino de fases más probable. Es Forward cambiando **suma por máximo** y guardando ψ para reconstruir el camino.

Componentes: δ = mejor puntaje de un camino que termina en i; ψ = "de dónde vine"; en logaritmos los productos se vuelven sumas.

En el código: HMM.viterbi(). Se usa en la proyección para contar cuántos pasos lleva la fase actual.

5.5 Problema 3: entrenamiento (Baum-Welch)

F12 Probabilidades a posteriori γ y ξ · ecs. 26–27 y 36–38

```
γ_t(i)   = P(q_t = S_i | O, λ) = α_t(i) β_t(i) / Σ_j α_t(j) β_t(j)
ξ_t(i,j) = P(q_t = S_i, q_{t+1} = S_j | O, λ) = α̂t(i) · a_ij · b_j(O_{t+1}) ·
β̂{t+1}(j)
γ_t(i)   = Σ_j ξ_t(i,j)
```

Qué hace: γ = probabilidad de estar en la fase i en el instante t; ξ = probabilidad de pasar de i a j entre t y t+1, ambas sabiendo todo el trazo.

En el código: dentro de HMM.fit() (gamma y la suma vectorizada de ξ).

F13 Reestimación con varias secuencias · ecs. 40b–c (+ sección V-B)

```
ā_ij  = Σ_k Σ_{t=1..T−1} ξ_t^(k)(i,j)  /  Σ_k Σ_{t=1..T−1} γ_t^(k)(i)

b̄_j(m) = Σ_k Σ_{t: O_t = v_m} γ_t^(k)(j)  /  Σ_k Σ_{t=1..T} γ_t^(k)(j)

Piso:  b̄_j(m) ← max(b̄_j(m), ε),  ε = 0.001,  y se renormaliza la fila
π se mantiene fijo en (1, 0, …, 0)
```

Qué hace: mejora el modelo usando "conteos esperados": transiciones esperadas de i a j entre transiciones esperadas que salen de i; veces esperadas en j viendo el símbolo m entre veces esperadas en j. Se repite hasta que la verosimilitud deja de crecer (máx. 30 iteraciones, tolerancia 10^{-4}). Garantía (ec. 42): $P(O \mid \bar{\lambda}) \geq P(O \mid \lambda)$.

Componentes: k = índice del ejemplo de entrenamiento (hasta 120 por letra); ε = piso para que ninguna dirección tenga probabilidad 0 (cada persona dibuja distinto).

En el código: HMM.fit(). Test: test_baum_welch_nunca_baja.

F14 Inicialización por segmentación uniforme (sección V-C)

```
Cada secuencia se divide en N tramos iguales; el tramo j alimenta al estado j:
b_j(k) = (nº de veces que aparece k en los tramos j + 0.5) / total del estado
j
L = T_promedio / N   (pasos por fase)        a_ii = clip(1 − 1/L, 0.05, 0.95)
```

Qué hace: da a Baum-Welch un punto de partida razonable (solo garantiza máximos locales).

En el código: HMM.init_from_segments().

F15 Número de estados por letra

```
N_c = clip( round( mediana(T) / 4 ), 4, 14 )
```

Qué hace: letras largas (m, w, ñ) tienen más fases que letras cortas (i, l, v); unas 4 flechas por fase.

En el código: training/train.py → numero_de_estados(). Resultado: i, l, v = 4 fases; b = 12.

F16 Duración de cada fase

```
Teórica (distribución geométrica):  P(d) = a_ii^(d−1) (1 − a_ii),     E[d] = 1 / (1 − a_ii)
Usada (medida en el entrenamiento):  d_j = (1/K) Σ_k Σ_t γ_t^(k)(j)
```

Qué hace: dice cuántos pasos suele durar cada fase. Se usa la versión medida porque la última fase es absorbente (a_NN = 1 → la fórmula teórica daría infinito).

En el código: self.durations, calculado en HMM.fit().

F17 Probabilidad de terminar η (mejora del 82.6 % al 88.6 %)

```
η_i = Σ_k γ_T^(k)(i)  /  Σ_k Σ_t γ_t^(k)(i)          (veces que se terminó en i / veces en i), con piso ε

Mientras se dibuja:     P(O_1:t | λ)          = Σ_i α_t(i)
Al levantar el lápiz:   P(O completo | λ)     = Σ_i α_T(i) · η_i
En logaritmos:          log P = −Σ log c_t + log Σ_i α̂T(i) η_i
```

Qué hace: distingue "esto puede ser el *comienzo* de la letra" de "esto puede ser la letra *completa*". Antes, una "c" terminada parecía el inicio de una "a" y perdía. Con η, la "a" casi nunca termina en su fase de "c", y la "c" gana.

En el código: self.end_probs, HMM.log_end(), Recognizer.posteriors(completo=...). Test: test_probabilidad_de_terminar.

5.6 Clasificación y proyección

F18 Bayes con prior uniforme (ranking de letras)

```
P(c | O) = P(O | λ_c) · P(c) / Σ_{k ∈ C} P(O | λ_k) · P(k),        P(c) = 1/27
En logaritmos (softmax):  P(c | O) = exp(ℓ_c − ℓ_max) / Σ_k exp(ℓ_k − ℓ_max),
ℓ_c = log P(O | λ_c)
Letra ganadora:  ĉ = argmax_c P(c | O)
```

Qué hace: convierte las 27 verosimilitudes en porcentajes que suman 100 %. Se resta ℓ_max para no desbordar exp(). El prior es uniforme para no favorecer letras con más muestras (q, i).

En el código: Recognizer.posteriors().

F19 Fase actual y pasos ya recorridos

```
s = argmax_i α̂t(i)                    (fase más probable ahora, según Forward)
h = nº de pasos al final del camino de Viterbi que están en la fase s
```

Qué hace: ubica en qué fase de la letra ganadora va el usuario y cuánto lleva en ella.

En el código: Recognizer.project().

F20 Dirección esperada de cada fase

```
u_k = (cos(2πk/16), −sin(2πk/16))          (vector unitario de la dirección k en pantalla)
Lápiz abajo si Σ_{k≥16} b_j(k) < 0.5
v_j = Σ_k b_j(k) · u_k / Σ_k b_j(k)         (sumando solo los símbolos del tipo de lápiz elegido)
```

Qué hace: resume la matriz B de cada fase en un solo vector: hacia dónde se mueve "en promedio" esa fase.

En el código: Recognizer.project() (pesos, vec).

F21 Proyección de la trayectoria futura

```
Pasos que faltan:  n_s = round(d_s − h),     n_j = round(d_j)  para j = s+1, …, N
Para cada paso:    p_{n+1} = p_n + STEP · v_j        (p_0 = posición actual del cursor)
```

Qué hace: desde el cursor, "camina" las fases que faltan de la letra ganadora, cada una durante su duración y en su dirección esperada. Se dibuja punteado (lápiz abajo) o muy tenue (lápiz arriba).

En el código: Recognizer.project() → Canvas2D.set_proyeccion().

F22 Histéresis (evitar parpadeo)

```
Cambiar la letra proyectada a ĉ  solo si  P(ĉ | O) > P(actual | O) + 0.10
```

Qué hace: si dos letras van casi empatadas, la línea punteada no salta de una a otra con cada movimiento.

En el código: View2D._actualizar(), MARGEN_CAMBIO = 0.10.

F23 Métricas de evaluación

```
top-1 = (nº de muestras de prueba donde la letra correcta queda 1.ª) / total
top-3 = (nº donde la correcta queda entre las 3 primeras) / total
Precisión al x % = top-1 usando solo el primer x % de los símbolos
```

En el código: training/evaluate.py (20 % de los datos separados para prueba).

6. Ejemplo resuelto a mano (para el tablero)

Dos letras de 2 fases, 3 símbolos (↓, →, ↑), observación **O = (↓, ↓, →)**.

```
π = [1, 0]      A = | 0.7  0.3 |      B_L = | 0.7 0.2 0.1 |   (fase 1: bajar)
                    | 0.0  1.0 |            | 0.1 0.6 0.3 |   (fase 2: derecha)
                                     B_Γ = | 0.2 0.7 0.1 |   (fase 1: derecha)
                                           | 0.6 0.3 0.1 |   (fase 2: bajar)
```

Forward (L)	**t=1 (↓)**	**t=2 (↓)**	**t=3 (→)**
α(1)	1·0.7 = 0.7	(0.7·0.7)·0.7 = 0.343	(0.343·0.7)·0.2 = 0.04802
α(2)	0	(0.7·0.3)·0.1 = 0.021	(0.343·0.3 + 0.021)·0.6 = 0.07434
P(O | λ_L) = 0.04802 + 0.07434			**0.12236**

Fuerza bruta: caminos 1-1-1 = 0.04802; 1-1-2 = 0.06174; 1-2-2 = 0.01260; total 0.12236 ✓.

Viterbi (L): δ_3 = [0.04802, 0.06174] → q_3^* = 2; $\psi_3(2)$ = 1; $\psi_2(1)$ = 1 → camino **1 → 1 → 2**, P^* = 0.06174.

Γ: α_3 = [0.01372, 0.01332], P(O | λ_Γ) = 0.02704.

Bayes: P(L | O) = 0.12236 / (0.12236 + 0.02704) = **81.9 %**; P(Γ | O) = 18.1 %.

Escalamiento (L): c = [1.4286, 1.9231, 2.9748]; $\hat{\alpha}_3$ = [0.3924, 0.6076]; $-\Sigma \log c_t = -2.1008 = \log 0.12236$ ✓.

Estos mismos números los comprueban automáticamente los tests (test_forward_ejemplo_a_mano, test_viterbi_ejemplo_a_mano, test_alpha_escalado_ejemplo_a_mano).

7. Parámetros y resultados

Parámetro	Valor	Motivo
Espacio del lienzo	0–1000, y hacia abajo	El mismo del dataset Glifo
Guías (ascendente, altura x, base, descendente)	180, 420, 700, 900	Las mismas de Glifo, para que el tamaño coincida
STEP (remuestreo)	25	1/40 del lienzo
K (direcciones) / M (símbolos)	16 / 32	16 con lápiz abajo + 16 con lápiz arriba
Δ (salto máximo)	2	Permite saltarse una fase en trazos cortos
N (fases)	4 a 14 según la letra	F15
ε (piso de B y η)	0.001	Evitar probabilidades 0
Iteraciones / tolerancia	30 / 10^{-4}	Convergencia de Baum-Welch
Muestras por letra (máx.)	120	Balancear letras con muchas muestras
Prueba	20 %	Datos que el modelo nunca vio
Margen de histéresis	0.10	F22

Dataset: glifo-dataset-2026-11-06.json, 5 711 muestras válidas (4 569 entrenamiento, 1 142 prueba).

Trazo dibujado	25 %	50 %	75 %	100 % (sin η)	100 % (con η)
top-1	29.9 %	50.3 %	70.5 %	82.6 %	**88.6 %**
top-3	58.2 %	73.9 %	87.0 %	92.1 %	**95.4 %**

Confusiones que quedan: h → n (17), b → p (8), r → n (8). Causa: solo se usan direcciones, no la altura donde empieza el trazo.

> El archivo exportado sin "detalles" no trae el autor; por eso la prueba se separó por muestra y no por persona, y los porcentajes pueden ser algo optimistas. Con el archivo "con detalles" la separación es por persona (más honesta).

8. Lo que falta

- Usar la altura del trazo (zona entre guías) para separar h/n y b/p.
- Ventana C (3D): esfera con rotación y zoom, dibujo sobre la superficie, coordenadas esféricas (θ, φ), entrenamiento 3D y proyección sobre la esfera.
- README final, ZIP al correo del docente y ensayo de la sustentación (Forward, Viterbi, Baum-Welch y proyección en el tablero).

9. Cómo ejecutar el programa en un computador nuevo

Instrucciones para Windows, partiendo de un computador que **solo tiene Python instalado**. Todos los comandos se escriben en **PowerShell**.

Paso 1 · Comprobar Python

```
python --version
```

Debe mostrar **Python 3.11 o superior**. Si sale un error o se abre la Microsoft Store, instala Python desde **python.org** y en el instalador marca la casilla **"Add python.exe to PATH"**. Cierra y vuelve a abrir PowerShell.

Paso 2 · Descargar el proyecto

Opción A (sin instalar nada más): en **github.com/lopezand2701/Estocasticos** pulsa el botón verde **Code → Download ZIP**. Descomprime el ZIP, por ejemplo en `Documentos\Proyecto`.

Opción B (si el computador tiene Git):

```
cd $env:USERPROFILE\Documents
git clone https://github.com/lopezand2701/Estocasticos.git Proyecto
```

Si el repositorio es privado, GitHub pedirá iniciar sesión (o usa la opción A con tu sesión abierta en el navegador).

Paso 3 · Entrar a la carpeta del proyecto

```
cd $env:USERPROFILE\Documents\Proyecto
dir
```

Debes ver `main.py`, `requirements.txt` y las carpetas `src`, `training`, `models`... Si descomprimiste un ZIP y ves otra carpeta dentro (p. ej. `Estocasticos-main`), entra en ella con `cd Estocasticos-main`.

Paso 4 · Crear y activar el entorno virtual

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Si PowerShell dice que la ejecución de scripts está deshabilitada, ejecuta una sola vez lo siguiente y vuelve a activar:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
.venv\Scripts\Activate.ps1
```

Sabrás que está activo porque la línea empieza con **(.venv)**. Cada vez que abras una terminal nueva debes volver a activarlo.

Paso 5 · Instalar las librerías

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python -c "import numpy, PySide6, pyqtgraph; print('OK')"
```

El último comando debe imprimir **OK**.

Paso 6 · Comprobar con los tests

```
pytest
```

Debe decir **13 passed**.

Paso 7 · ¿Ya están los modelos entrenados?

```
dir models\2d
```

Si aparece `hmm_2d.json`, **salta al paso 10**. Si no aparece, sigue con los pasos 8 y 9.

Paso 8 · Descargar el dataset de Glifo y moverlo al proyecto

En la página de Glifo, sección **"Exportar para entrenar"**, elige **"Todos los grupos"** y pulsa **"Descargar con detalles"** (o "Descargar dataset .json"). Luego:

```
Move-Item "$env:USERPROFILE\Downloads\glifo-dataset-*.json" "data\raw\"
dir data\raw
```

Debe aparecer el archivo `glifo-dataset-…json` (unos 15 MB).

Paso 9 · Entrenar y evaluar

```
python -m training.train
python -m training.evaluate
```

El entrenamiento muestra una línea por letra (p. ej. `[a] muestras=120 N=11 iter=15 …`) y guarda `models\2d\hmm_2d.json`. La evaluación muestra la precisión (≈ 88 % top-1 con el trazo completo).

Paso 10 · Abrir la aplicación

```
python main.py
```

En el menú elige **Modo 2D** y dibuja una letra con el mouse. Al soltar el botón, la letra se evalúa como completa.

Resumen de comandos (copiar y pegar)

```
cd $env:USERPROFILE\Documents\Proyecto
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
python -m training.train          # solo si no existe models\2d\hmm_2d.json
python -m training.evaluate
python main.py
```

Para no tener que entrenar en el otro computador

Desde tu computador actual, sube a GitHub los modelos y el dataset (el .gitignore no los excluye):

```
git add models data/raw
git commit -m "Modelos entrenados y dataset"
git push
```

Problemas frecuentes

Mensaje	Solución
`python no se reconoce como comando`	Reinstalar Python marcando "Add python.exe to PATH" y abrir una terminal nueva.
`la ejecución de scripts está deshabilitada`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError: No module named 'numpy'` (o PySide6)	El entorno no está activo: `.venv\Scripts\Activate.ps1` y repetir `pip install -r requirements.txt`.
`ModuleNotFoundError: No module named 'src'` o `'training'`	Estás en otra carpeta: haz `cd` a la carpeta donde está `main.py`. Usa `python -m training.train` (con `-m`).
`No hay archivos .json en data/raw`	Falta el dataset: paso 8.
Ventana "Faltan los modelos"	Falta `models\2d\hmm_2d.json`: paso 9.
Textos cortados en la ventana	Asegúrate de tener la última versión de `src/gui` (corregida para escala 125–150 %).

En macOS o Linux los comandos son iguales salvo: `python3` en vez de `python`, activar con `source .venv/bin/activate` y mover archivos con `mv ~/Downloads/glifo-dataset-*.json data/raw/`.